

长短期记忆网络 (LSTM) 复习文档

你的 AI 助手 Gemini

2025 年 9 月 2 日

目录

1	通俗解释	2
2	数学原理与分析过程	2
2.1	第一步：遗忘门	3
2.2	第二步：输入门	3
2.3	第三步：更新细胞状态	3
2.4	第四步：输出门	4
3	代码实现 (Python - TensorFlow)	4

1 通俗解释

想象一下，你在读一本很长的小说。当你在读后面的章节时，你肯定需要记住前面出现过的主要人物、关键情节，不然你就会看得云里雾里。但是，你需不需要记住几百页前主角早餐吃了什么？很可能不需要。我们的大脑就很擅长做这件事：**记住重要的信息，忘记无关紧要的细节。**

传统的循环神经网络 (Recurrent Neural Network, RNN) 就像一个记性不太好的人，特别是对于很久以前的事情，它很容易“遗忘”，这在学术上被称为“长期依赖问题” (Long-Term Dependency Problem) 或“梯度消失问题” (Vanishing Gradient Problem)。当信息需要在很长的时间序列中传递时，RNN 就会把最早期的重要信息给忘了。

为了解决这个问题，研究人员设计出了更精密的“大脑”——**长短期记忆网络 (LSTM)**。LSTM 可以看作是 RNN 的一个升级版，它内部设计了非常精妙的“门控结构” (Gate Structure)，就像我们记忆系统里的开关一样，来控制信息的流动。

这个“门”主要有三个：

1. **遗忘门 (Forget Gate)**：它负责决定哪些旧的信息应该被“扔掉”或“忘记”。比如，小说换了一个新的场景，那么上一个场景的一些次要细节就可以忘记了。
2. **输入门 (Input Gate)**：它负责决定当前这个时间点，有哪些新的信息是重要的，需要被“记下来”。比如，小说里出现了一个新的关键人物，那他的信息就需要被记住。
3. **输出门 (Output Gate)**：它负责决定在当前这个时间点，我们需要从记忆中提取哪些信息来做出判断或预测。比如，根据你记住的所有情节，来推测接下来会发生什么。

通过这三个门的协同工作，LSTM 就能够有效地学习到哪些信息是需要长期记住的，哪些信息是暂时的，从而解决了传统 RNN 的“健忘”问题。这让它在处理像时间序列预测（比如股票价格）、自然语言处理（比如机器翻译、文本生成）等需要处理长序列数据的任务时，表现得非常出色。

2 数学原理与分析过程

现在，我们深入到 LSTM 的内部，看看这些“门”在数学上是如何工作的。一个 LSTM 单元的核心是**细胞状态 (Cell State)**，用 C_t 表示。你可以把它想象成一条传送带，信息可以在上面一直传递下去，而我们刚刚提到的三个“门”就是控制这条传送带上信息增加或减少的开关。

假设在时间步 t ，我们的输入是 x_t ，前一个时间步的隐藏状态是 h_{t-1} ，前一个时间步的细胞状态是 C_{t-1} 。LSTM 的目标是计算出当前时间步的隐藏状态 h_t 和细胞状态 C_t 。

2.1 第一步：遗忘门

首先，LSTM 需要决定从上一个细胞状态 C_{t-1} 中丢弃什么信息。这个决定由“遗忘门”做出。它会查看 h_{t-1} 和 x_t ，然后为 C_{t-1} 中的每个数字输出一个在 0 到 1 之间的数值。1 代表“完全保留”，0 代表“完全丢弃”。它的计算公式如下：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

其中：

- f_t 是遗忘门的输出向量。
- σ 是 Sigmoid 激活函数，其输出值域是 $(0, 1)$ 。
- W_f 是遗忘门的权重矩阵， b_f 是偏置项。
- $[h_{t-1}, x_t]$ 表示将前一个隐藏状态和当前输入拼接成一个更长的向量。

2.2 第二步：输入门

接下来，我们要决定在细胞状态中存放哪些新信息。这一步分为两部分：一个叫做“输入门”的 Sigmoid 层决定我们要更新哪些值；然后，一个 $\tanh$ 层创建一个新的候选值向量 $\tilde{C}_t$ ，这个向量可以被加到细胞状态中。它们的计算公式如下：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

其中：

- i_t 是输入门的输出，决定了哪些新信息需要被更新。
- $\tilde{C}_t$ 是候选的细胞状态值。
- $\tanh$ 是双曲正切激活函数，其输出值域是 $(-1, 1)$ 。

2.3 第三步：更新细胞状态

现在，我们可以更新旧的细胞状态 C_{t-1} 为新的细胞状态 C_t 了。我们用旧状态 C_{t-1} 乘以 f_t 来忘记我们决定要忘记的东西。然后，我们再加上 $i_t * \tilde{C}_t$ ，这就是我们决定要加入的新信息。

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

其中 $\odot$ 表示哈达玛积 (Hadamard Product)，也就是两个向量对应元素相乘。这个公式非常直观：忘记一部分旧的记忆 ($f_t \odot C_{t-1}$)，然后学习一部分新的记忆 ($i_t \odot \tilde{C}_t$)。

2.4 第四步：输出门

最后，我们需要决定要输出什么。这个输出将基于我们的细胞状态，但会是一个“过滤后”的版本。首先，我们运行一个 Sigmoid 层来决定细胞状态的哪个部分将输出。然后，我们把细胞状态通过 tanh 进行处理，并将其与 Sigmoid 门的输出相乘，这样我们就得到了我们只想输出的那部分。公式如下：

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

其中：

- o_t 是输出门的输出。
- h_t 就是当前时间步最终的隐藏状态，它既作为当前时间步的输出，也会作为下一个时间步的输入 h_t 。

3 代码实现 (Python - TensorFlow)

下面我们使用 TensorFlow/Keras 框架来实现一个简单的 LSTM 模型。任务与之前类似：根据过去一段时间的正弦函数值，预测下一个值。

```
1 import numpy as np
2 import tensorflow as tf
3 from tensorflow.keras.models import Sequential
4 from tensorflow.keras.layers import LSTM, Dense
5 import matplotlib.pyplot as plt
6
7 # 1. 准备数据
8 # 假设我们有一段时间序列数据，例如 sin 函数
9 def generate_data(n_points=1000, sequence_length=10):
10     """生成数据"""
11     data = np.sin(np.linspace(0, 100, n_points))
12     X, y = [], []
13     for i in range(len(data) - sequence_length):
14         X.append(data[i:i+sequence_length])
15         y.append(data[i+sequence_length])
16
17     # 将数据 reshape 成 LSTM 需要的格式 [samples, timesteps, features]
18     X = np.array(X).reshape(-1, sequence_length, 1)
19     y = np.array(y)
20
21     return X, y
22
```

```

23 sequence_length = 20
24 X, y = generate_data(sequence_length=sequence_length)
25
26 print("X shape:", X.shape) # (980, 20, 1)
27 print("y shape:", y.shape) # (980,)
28
29
30 # 2. 构建 LSTM 模型
31 model = Sequential([
32     # input_shape=(timesteps, features)
33     # timesteps: 序列长度
34     # features: 每个时间步的特征数
35     LSTM(units=64, input_shape=(sequence_length, 1), return_sequences=True)
36     ,
37     LSTM(units=32),
38     Dense(units=1)
39 ])
40
41 # 3. 编译模型
42 model.compile(optimizer='adam', loss='mean_squared_error')
43
44 # 打印模型结构
45 model.summary()
46
47 # 4. 训练模型
48 history = model.fit(X, y, epochs=20, batch_size=32, verbose=1)
49
50
51 # 5. 模型预测与评估
52 # 使用训练集的最后一个序列进行预测
53 last_sequence = X[-1:]
54 predicted_value = model.predict(last_sequence)
55
56 print("\n--- 预测 ---")
57 print(f"真实值: {y[-1]:.4f}")
58 print(f"预测值: {predicted_value[0][0]:.4f}")
59
60 # 可视化训练损失
61 plt.figure(figsize=(10, 6))
62 plt.plot(history.history['loss'])
63 plt.title('Model loss during training')
64 plt.ylabel('Loss')
65 plt.xlabel('Epoch')
66 plt.legend(['Train'], loc='upper right')

```

```
67 plt.grid(True)
68 plt.show()
```

Listing 1: 使用 TensorFlow/Keras 构建 LSTM 模型